VIRGO

SLM V1.0

Building a 120M-Parameter Transformer Language Model from Scratch

A Self-Learning Engineering Project

"Not Everybody Needs The Blueprint."

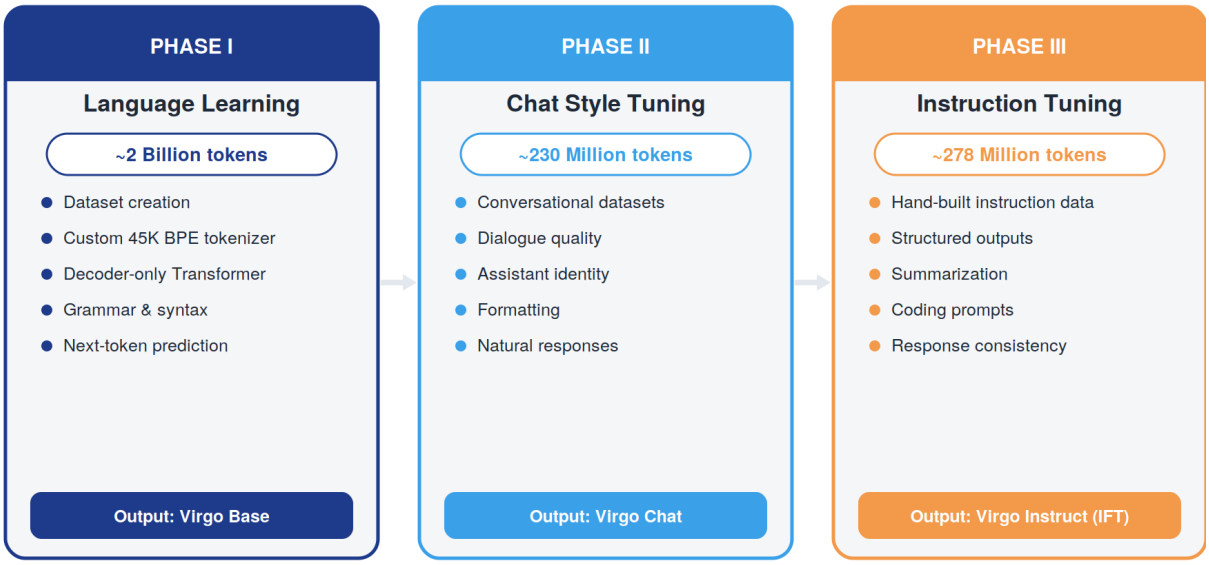
Punit Kumar Kashyap
First-Year Undergraduate, Department of Engineering Physics
Indian Institute of Technology Dharwad
2026

01 Abstract

Virgo SLM V1.0 is a 120-million-parameter decoder-only Transformer language model that I designed, trained, and fine-tuned entirely on my own, as a first-year undergraduate at IIT Dharwad. This report is not a research paper — it is an honest account of an engineering journey: the choices I made, the mistakes I learned from, and the small model that resulted from nearly a year of curiosity-driven work.

Virgo was built in three phases. Phase I taught the model language itself — grammar, vocabulary, and next-token prediction — over roughly 2 billion tokens, producing Virgo Base. Phase II reshaped it into a conversational assistant, Virgo Chat, using around 230 million tokens of dialogue data. Phase III, the most personally demanding stage, used a 278-million-token instruction dataset that I hand-curated over several months to produce Virgo Instruct, the final IFT model.

Virgo is small by industry standards, and it knows it. It is not a competitor to GPT, Gemini, or Claude — it was never meant to be. It is a proof that a single student, with a laptop, patience, and enough failed training runs, can build something that actually works, actually generates language, and actually teaches more than any textbook could.



02 Why I Started Virgo

Somewhere in my first semester at IIT Dharwad, I had read enough papers on attention mechanisms to explain them on a whiteboard — but I still couldn't picture what actually happened inside a Transformer when a prompt went in and a sentence came out. I could recite the equations for self-attention. I could not tell you, with confidence, why a specific token was chosen over another during generation.

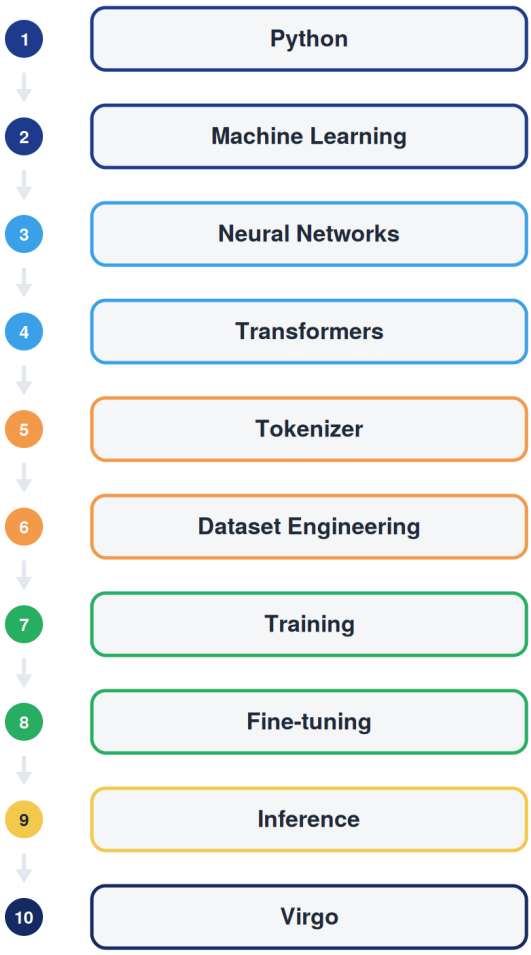
That gap between reading and understanding is what started Virgo. I did not want another summarized version of 'Attention Is All You Need.' I wanted to write the embedding layer myself, watch the loss curve refuse to go down, and figure out why. I wanted to build a tokenizer and watch it fail on an edge case I hadn't considered. Papers explain what works. They rarely explain what it feels like when it doesn't.

“Not Everybody Needs The Blueprint.”

That line became the project's tagline because it is exactly what Virgo represents to me. There is no single official path to learning how language models work. I did not wait for a lab position, a mentor, or a perfect curriculum. I opened a blank Python file and started with a bigram model, then a simple attention head, then a full decoder block — building understanding one broken training run at a time.

03 My Learning Journey

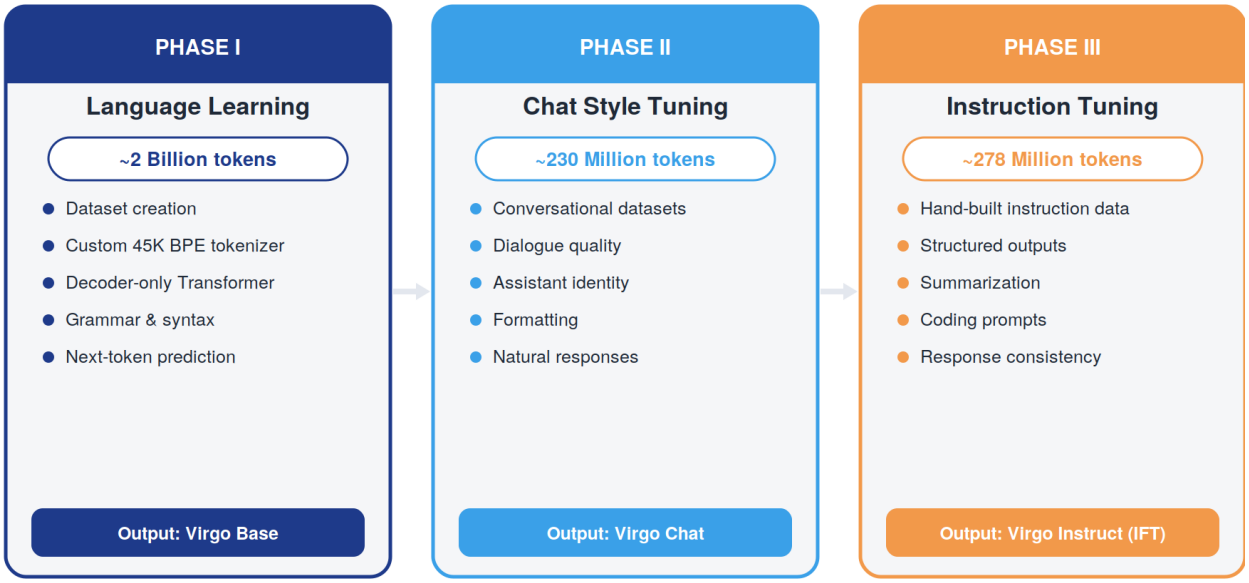
Before Virgo could exist, I had to build the foundation underneath it. This was not a fast process — it stretched across most of my first year, layered on top of regular coursework in Engineering Physics. The path below is the honest sequence of topics I worked through, each one unlocking the next.



The jump from 'Neural Networks' to 'Transformers' was the hardest step in this entire chain. Everything before it — perceptrons, backpropagation, gradient descent — had a fairly intuitive mechanical story. Self-attention did not, at first. It took re-implementing scaled dot-product attention from scratch, by hand, with a toy 4-token example on paper, before it actually clicked.

04 Virgo Development Roadmap

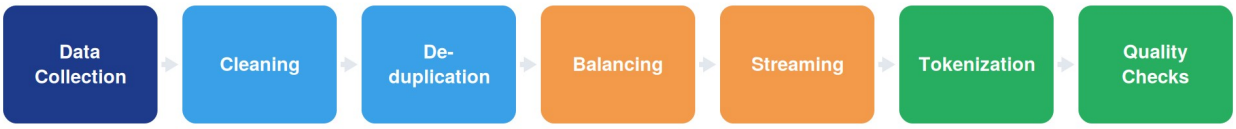
Virgo was never built in one pass. I split the project into three deliberate phases, each with its own dataset, its own objective, and its own set of failures. This structure — pretrain, then chat-tune, then instruction-tune — mirrors how modern language models are trained, and building it this way let me isolate exactly what each stage contributes.



Phase III took the longest by far. Training compute for that stage was a fraction of Phase I, but the dataset behind it consumed months of manual work — writing prompts, generating and cleaning responses, balancing task categories like summarization, extraction, and structured formatting so the model wouldn't overfit to any single instruction style.

05 Dataset Engineering

If there is one lesson Virgo taught me above all others, it is that a language model is only as good as the data pipeline feeding it. I spent more hours cleaning, deduplicating, and balancing text than I spent writing the model architecture itself.

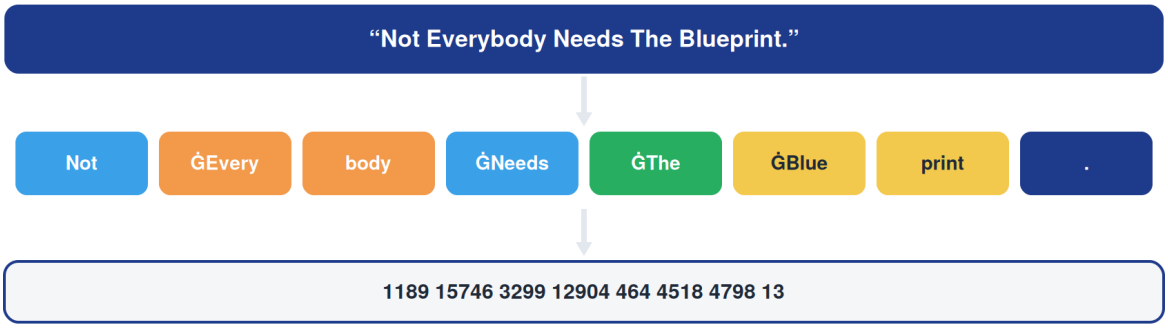


What each stage actually involved

- Collection: pulling from open web-text corpora, book-style writing, and public conversational datasets, kept within permissive licenses.
- Cleaning: stripping HTML remnants, boilerplate, broken Unicode, and duplicate whitespace that silently inflated token counts.
- Deduplication: near-duplicate documents were quietly wrecking my early loss curves by letting the model memorize instead of generalize.
- Balancing: making sure no single domain (news, code, dialogue) dominated the mixture and skewed the model's 'personality.'
- Streaming: chunking cleaned text into fixed-length sequences that could be streamed during training without loading everything into memory at once.
- Quality checks: sampling batches by hand, again and again, because automated filters missed things a five-second human read caught instantly.

06 Tokenizer

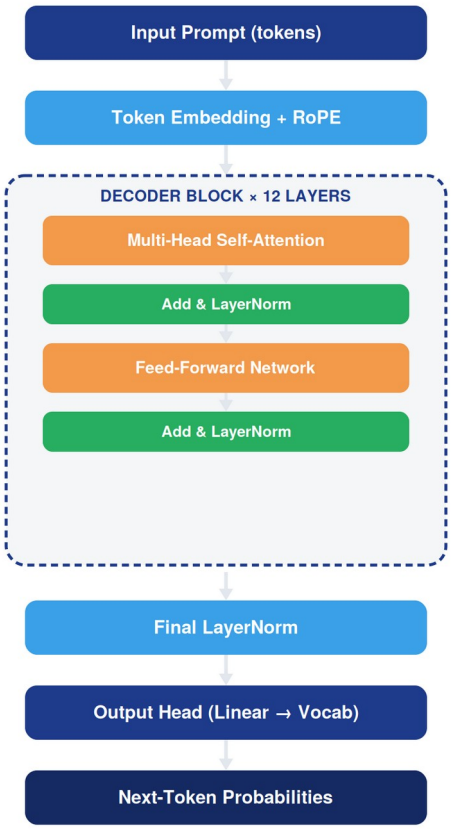
Virgo uses a custom Byte-Level BPE (Byte Pair Encoding) tokenizer with a 45,000-token vocabulary, trained from scratch on the same corpus used for pretraining rather than reused from an existing model. Byte-level BPE was the right choice for a project like this: it never produces an out-of-vocabulary token, since in the worst case it falls back to raw bytes.



Training the tokenizer myself, rather than importing GPT-2's or a similar off-the-shelf vocabulary, meant the merge rules actually reflected the vocabulary and style of my own dataset. It also meant debugging genuinely strange failures — like special tokens for chat formatting (system, user, assistant markers) colliding with byte-pair merges until I reserved them explicitly outside the trainable vocabulary.

07 Transformer Architecture

Virgo is a decoder-only Transformer, architecturally close to the GPT family, with roughly 119.6 million parameters across 12 layers, a 768-dimensional hidden size, and 12 attention heads per layer. I chose RoPE (Rotary Positional Embeddings) over learned absolute position embeddings, mainly because implementing it forced me to understand rotation matrices and relative position encoding at a level that reading about it never had.



Each of the 12 decoder blocks follows the same structure: multi-head self-attention with a causal mask (so a token can never see the future), followed by an Add & LayerNorm residual step, then a position-wise feed-forward network, followed by another Add & LayerNorm. The residual connections were, without exaggeration, the single architectural detail that made deep training stable — the first version of Virgo without properly scaled residuals refused to train past a few hundred steps.

Core specifications

Specification	Value
Parameters	≈ 119.6 Million
Layers	12 decoder blocks
Hidden size	768
Attention heads	12
Positional encoding	RoPE (Rotary)
Tokenizer	Byte-Level BPE, 45K vocabulary
Context length	Fixed-length training sequences

08 Training Journey

Training Virgo Base on ~2 billion tokens on consumer-grade hardware meant every optimization technique mattered. I relied on mixed-precision training (fp16/bf16) to fit larger batches into limited VRAM, gradient accumulation to simulate a bigger effective batch size than memory allowed, and frequent checkpointing because I genuinely could not trust any single training run to survive to completion.

> **ENGINEERING NOTE**
>
> Learning-rate warmup was not optional. My first few runs used a flat learning rate from step one, and the loss would spike and never fully recover. A short linear warmup followed by cosine decay fixed this almost immediately.

Watching the training loss curve become one of the more meditative parts of this project. In the early pretraining steps, the model's outputs were pure noise — repeated characters, no word boundaries. Somewhere around a few hundred million tokens in, actual English words began appearing. By the end of Phase I, Virgo Base could complete a sentence with correct grammar, even if the content was often nonsensical.

09 Challenges & Lessons Learned

This is the chapter I almost didn't want to write, because it means admitting how many times Virgo simply broke. But this is also the most useful chapter for anyone attempting something similar — because every one of these failures taught me more than a successful run ever did.

CHALLENGE: CUDA Out-of-Memory

My first architecture attempt used a batch size that looked reasonable on paper but ignored activation memory during backpropagation. Gradient checkpointing and gradient accumulation solved this, at the cost of slower steps.

CHALLENGE: Silent Tokenizer Bugs

An early version of my tokenizer merged a newline character into unrelated word pairs due to a preprocessing bug, quietly corrupting a portion of every document. It took a full week of debugging generation quality before I found the root cause.

CHALLENGE: Checkpoint Corruption

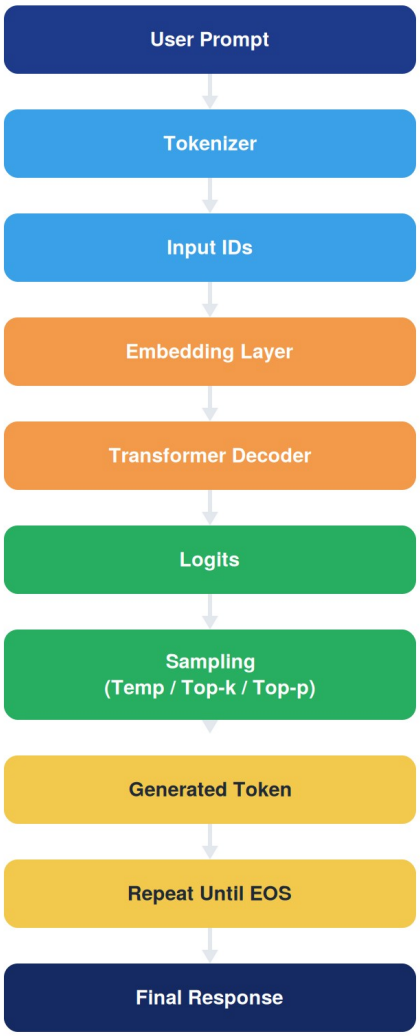
Two separate training runs were lost to interrupted checkpoint writes after unexpected shutdowns. I now write checkpoints to a temporary file first and only rename them into place after the write finishes completely.

LEARNING: Dataset Bugs Hide as Model Bugs

More than once, I spent hours tuning hyperparameters to fix a problem that was actually a mislabeled or duplicated data shard. Now, whenever a model behaves oddly, I check the data before I touch the architecture.

10 Virgo Inference Cycle

Every response Virgo produces follows the same repeating cycle, one token at a time. Understanding this loop end-to-end — and being able to trace a bug to any single stage in it — was one of the most valuable outcomes of this whole project.

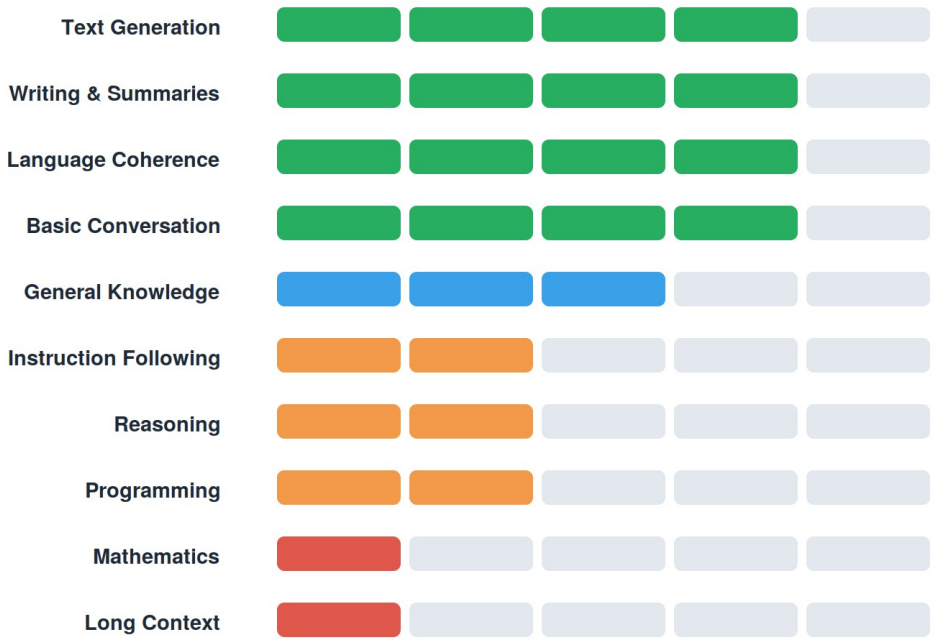


Decoding strategy

Virgo samples the next token using a combination of temperature scaling, top-k filtering, and top-p (nucleus) sampling rather than always picking the single highest-probability token. Temperature reshapes the probability distribution before sampling — lower values make the model more deterministic and repetitive, higher values make it more diverse and occasionally incoherent. Top-k restricts sampling to the k most likely tokens, and top-p restricts it to the smallest set of tokens whose cumulative probability crosses a threshold. In practice, combining moderate temperature with top-p sampling gave the most natural-sounding Virgo responses.

11 Capability Assessment

Being honest about what Virgo can and cannot do is more valuable to me than inflating its abilities. The dashboard below reflects real, observed behavior from Virgo Instruct across dozens of manual test prompts — not a benchmark score.



Virgo currently follows a strict prompt → response interaction model. It does not yet support persistent memory across turns, web search, tool use, image generation, file creation, or genuinely long-context conversations. These are not oversights — they are deliberate scope boundaries for a V1.0 project built by one person in one year.

12 Current Limitations

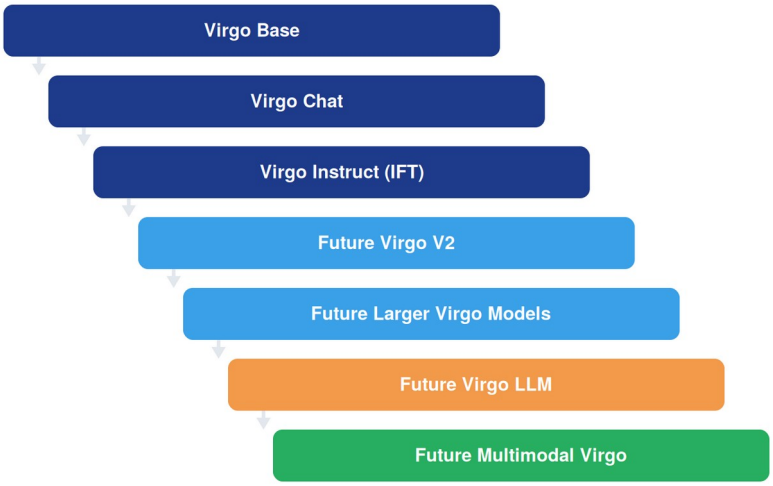
Virgo is, and should be read as, an early-stage 120-million-parameter Small Language Model. At this scale, several limitations are expected rather than surprising:

- Reasoning and multi-step instruction following remain the weakest capabilities, and are an active area for future work.
- Programming ability is limited to short, simple snippets — Virgo struggles with debugging or multi-file logic.
- Mathematical reasoning beyond basic arithmetic is unreliable and should not be trusted.
- Hallucinations occur, particularly on factual or niche knowledge questions outside the training distribution.
- Long, multi-turn conversations degrade in coherence due to the model's limited context window.

None of this is discouraging to me — if anything, it's the clearest possible roadmap for Virgo V2. A model that already does everything perfectly would leave nothing left to learn from.

13 Future Vision

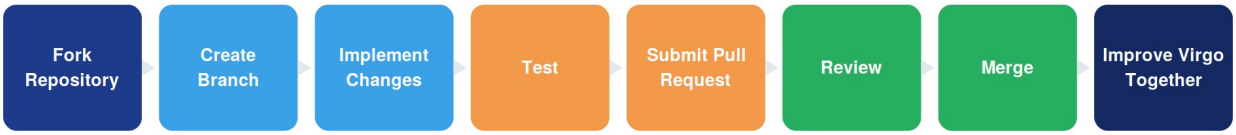
Virgo Instruct is a milestone, not a destination. I think of it as the first working generation of a family of models I intend to keep building and scaling as I learn more.



Future work will focus on stronger reasoning, better instruction following through improved and larger datasets, longer context windows, retrieval-augmented generation, persistent memory, tool use, web search, and eventually image understanding and multimodal capability — all while trying to keep the model efficient enough to train without a data-center budget.

14 Join the Virgo Community

Virgo is, and will remain, an open-source learning project. It was built by one student without a research lab, and it will grow faster and better with more curious hands on it. Contributions of any size are genuinely welcome — from a dataset fix to a full training-efficiency improvement.



Ways to contribute

- Dataset improvements, cleaning, and new instruction categories
- Documentation, tutorials, and educational explainers for beginners
- Model evaluation, benchmarking, and honest capability testing
- Bug fixes and training-efficiency optimizations
- New feature development and research ideas for Virgo V2

15 Final Author Message

When I started this project, I could not have told you what a residual connection was for. I definitely could not have told you what a CUDA out-of-memory error looked like at 2 a.m., or how it feels to lose a three-day training run to a single unhandled exception during checkpoint saving. Virgo taught me both, along with a hundred smaller lessons in between.

This project is not impressive because Virgo writes better than GPT-4 — it doesn't, and it was never meant to. It is meaningful to me because every single component in it, from the tokenizer's merge rules to the attention masks to the sampling logic, is something I understood well enough to build, break, and rebuild myself.

If you are a student reading this and thinking about starting something similar: you do not need the perfect roadmap, the perfect GPU, or the perfect dataset. You need a blank file, the willingness to watch a loss curve fail to converge a dozen times, and the patience to debug the thirteenth. Start before you feel ready. You will learn the rest on the way.

"Not Everybody Needs The Blueprint."

— Punit Kumar Kashyap
Engineering Physics, IIT Dharwad